Q1 – Problem

- What is the output of this C code?
- #include <stdio.h>
- int main() {
- int i = 0, j = 0;
- while (I1: i < 2) {
- i++;
- while (j < 3) {
- printf("loop\n");
- goto I1;
- }
- }
- }

Options:
A. loop loop loop loop
B. loop loop
C. Compile time error
D. Infinite loop

Q1 – Explanation

- The goto label is expected to be used outside conditional areas.
- In the given code, the compiler fails to identify I1 as a label and considers it an undeclared identifier.
- Result: Compile-time error.

Q1 – Answer

- Answer: Option C

Q2 – Problem

- What is the output of this code?
- #include<stdio.h>
- int main() {
- static int i = 5;
- if (--i) {
- printf("%d ", i);
- main();
- }
- }

Options:
A. 4 3 2 1
B. 4 4 4 4
C. 0 0 0 0
D. 1 2 3 4

Q2 – Explanation

- i is a static variable, initialized to 5.
- The main() function is recursively called, decrementing i each time.
- The recursion continues until i becomes 0, printing the values 4,3,2,1.
- Result: 4 3 2 1.

Q2 – Answer

- Answer: Option A

Q3 – Problem

- Consider the below given fragment of code.

```
int f(int &x, int c) {
    c = c - 1;
    if (c == 0) return 1;
    x = x + 1;
    return f(x, c) * x;
}
```

Options:
A. 6561
B. 161051
C. 55440
D. 3024

- First parameter by reference, second by value. If p=5, find f(p,p)

Dry Run

- int f(int &x, int c) c = c-1, x = x+1
- F(5,5) -> f(6, 4) * 9

- F(6,4) → f (7, 3) * 9
- F(7,3) → f(8, 2)*9
- F(8,2) -> f(9, 1)*9
- F(9,1) → f(10, 0)*x → 1

- 1*9*9*9*9 → 6561

Q3 – Explanation

- x is passed by reference, so updated across recursive calls.
- Recursively calls f(x,c), multiplying by x.
- Recursion continues until c=0.
- Result: 6561.

Q3 – Answer

- Answer: Option A

Q4 – Problem

- Comment on the output of this C code:

```
#include <stdio.h>

int main() {
    int x = 3, i = 0;
    do {
        x = x++;
        i++;
    } while (i != 3);
    printf("%d\n", x);
}
```

Options:
A. 5
B. 6
C. 3
D. Undefined behaviour

Q4 – Explanation

- Post-increment x++ returns original value.
- x = x++ assigns old value to x.
- Loop runs 3 times, x remains unchanged.
- Result: 3.

•x = x++ → post-increment returns old value but increments
•internally.
•The assignment overwrites increment, leaving x unchanged.
•After 3 loops, x is still 3.

Q4 – Answer

- Answer: Option C

Q5 – Problem

- What will be the value of s if n=127?

```
Read n
i = 0, s = 0
Function Sample(int n)
   while (n > 0)
      r = n % 10
      p = 8^i
      s = s + p * r
      i++
      n = n / 10
   End While
   Return s
End Function
```

Options:
A. 27
B. 187
C. 87
D. 120

Q5 – Explanation

- Processes each digit starting from least significant.
- For each digit, calculate 8^i * digit, add to s.
- Result: 87.

- Iter1: n=127 → r=7 → s=0+1*7=7 → i=1 → n=12.
- Iter2: n=12 → r=2 → s=7+(8*2)=23 → i=2 → n=1.
- Iter3: n=1 → r=1 → s=23+(64*1)=87 → n=0.

Q5 – Answer

- Answer: Option C

Q6 – Problem

- Output if limit=6?

```
Read limit
n1 = 0, n2 = 1, n3 = 1, count = 1
while (count <= limit)
    count = count + 1
    n3 = n1 + n2
    n1 = n2
    n2 = n3
    print n3

End While
```

Options:
A. 1235813
B. 12358
C. 123581321
D. 12358132

Fibonacci Series

0,1,1 , 2, 3,5, 8, 13, 21

Q6 – Explanation

- Loop generates Fibonacci numbers updating n1,n2,n3.
- Loop runs 6 times, prints first 6 Fibonacci numbers.
- Result: 1235813.

  - Iter1: print1 → n3=1.
  - Iter2: print1 → n3=2.
  - Iter3: print2 → n3=3.
  - Iter4: print3 → n3=5.
  - Iter5: print5 → n3=8.
  - Iter6: print8 → n3=13.

  → Output: 1 1 2 3 5 8 → 1235813.

Q6 – Answer

- Answer: Option A

Q7 – Problem

- Value of even_counter if number=2630?

```
Read number
Function divisible(number)
   even_counter = 0
   num_remainder = number
   while (num_remainder)
      digit = num_remainder % 10
      if digit != 0 AND number % digit == 0
         even_counter = even_counter + 1
      End If
      num_remainder = num_remainder / 10
   End While
End Function
```

Options:

A. 2

B. 3

C. 1

D. 0

Q7 – Explanation

Digits: 0,3,6,2.

- d=0 → skip.
- d=3 → 2630%3=2 → not divisible.
- d=6 → 2630%6=0 → not div
- d=2 → 2630%2=0 → count=1

Q7 – Answer

- Answer: Option C

Q8 – Problem

- Output of pseudocode: m=9, n=6

```
Input m = 9, n = 6
m = m + 1
n = n - 1
m = m + n
if (m > n)
    print m
else
    print n
```

Options:
A.6
B.5
C.10
D.15

Q8 – Explanation

- After operations: m=10, n=5
- m>n → print m.
- Result: 10.

Q8 – Answer

- Answer: Option D

Q9 – Problem

- Output pseudocode: f=6, g=9, sum=0

```
Input f = 6, g = 9, sum = 0
Integer n
if (g > f)
   for (n = f; n < g; n = n + 1)
      sum = sum + n
End for
else
   print error message
print sum
```

Options:
A.21
B.15
C.9
D.6

Q9 – Explanation

- Loop adds 6+7+8=21.
- Result: 21.

Q9 – Answer

- Answer: Option A

Q10 – Problem

Consider the following piece of code. What will be the space required for this code?

```
int sum(int A[], int n) {
    int sum = 0, i;
    for (i = 0; i < n; i++)
        sum = sum + A[i];
    return sum;
}
```

Options:

A. 2n+8

B.2n+4

C.2n+2

D.2n

Q10 – Explanation

- Array A: n integers = 2n bytes.
- Local vars sum + i = 8 bytes.
- Total = 2n+8.

Q10 – Answer

- Answer: Option A

Q11 – Problem

- Output pseudocode: x=5, y=x

```
Input x = 5
y = x
while x > 0
    x = x - 1
    y = y + x
End while
print y
```

Options:
A.15
B.20
C.10
D.25

Q11 – Explanation

- x=5,y=5
- Loop iterations: x=4→y=9,x=3→y=12,x=2→y=14,x=1→y=15,x=0 ends
- Result: 15

Q11 – Answer

- Answer: Option A

Q12 – Problem

- Find output:

Input n = 4, fact = 1

for i = 1 to n

    fact = fact * i

End for

print fact

Options:
A.16
B.24
C.12
D.8

Q12 – Explanation

- Loop 1*2*3*4=24
- Result: 24

Q12 – Answer

- Answer: Option B

Q14 – Problem

- Output pseudocode: n=5,sum=0

```
Input n = 5
sum = 0
for i = 1 to n
    sum = sum + i*i
End for
print sum
```

Options:
A.55
B.50
C.45
D.60

Q14 – Explanation

- Sum squares 1+4+9+16+25=55
- Result: 55

Q14 – Answer

- Answer: Option A